

CS220: Lab#11

- All problems must be solved using MIPS assembly language programs. The programs should be run on the SPIM simulator. Type your program using any Linux editor. To start the simulator, open a terminal and type `qtspim`. To load your program into the simulator, *File*→*Reinitialize and Load File*. To run your program, *Simulator*→*Run/Continue*. For debugging, if you want to run one instruction at a time and examine the register values (visible in the left pane), *Simulator*→*Single Step* and then press F10 to execute the next instruction.
- Sections A.9 and A.10 from Appendix A of Patterson and Hennessy have all details regarding SPIM. You will need the MIPS instructions, system call table, and the assembler directives to write the programs.
- Use of pseudo-instructions supported by SPIM is allowed.
- When you enter a value from keyboard, make sure to press “ENTER” after entering the value; otherwise the system call layer will not accept the input. As a result, when entering an array of values, please enter them one per line.
- Each question carries two marks.

1. The file `q1_input.txt` contains a list of positive integers (one per line) terminated by 0 (i.e., the last integer is 0). Write a MIPS program to read the integers from the file and compute a histogram of the positive values which show the number of input integers in the following 11 ranges: `[1, 10]`, `[11, 20]`, `[21, 30]`, ..., `[91, 100]`, `> 100`. Print the computed histogram on the monitor. The expected output is given below. Your program should produce this output exactly in the given format.

```
Histogram:
[1, 10]: 38
[11, 20]: 32
[21, 30]: 51
[31, 40]: 44
[41, 50]: 42
[51, 60]: 44
[61, 70]: 41
[71, 80]: 36
[81, 90]: 44
[91, 100]: 42
> 100: 586
```

Note: To read from a file, you first need to open the file. The open system call requires the pointer to the string storing the complete path of the file in register `$a0`. To find the complete path of the file, go to the directory (using `cd` command) containing the file and then type the command `pwd`. The open system call returns a file descriptor (a non-negative integer id for the file) in register `$v0` (the book mistakenly mentions register `$a0`), which should be used to read the file. The read system call is used to read from a file. You should read one byte at a time. The returned byte is the ASCII value of the character read from the file. You can assume that the file contains only non-negative integers. So, the read byte will have value corresponding to the ASCII codes of the digits 0 to 9 or `\n`. You will have to construct the integer values from the read bytes. As an example, if the first line of the file

contains the number 12 followed by `\n`. The read bytes will have values 49, 50, 10 in decimal. You will detect the end of a line when you read a value 10. You will use the remaining values in a line to construct the integer. Remember to close the file before returning from `main`.

2. Write a MIPS program featuring a recursive procedure to solve a given equation $f(x) = 0$ in a single variable x using the bisection method. The function f should be evaluated for a given x in a separate function so that your implementation can be tested for different functions easily. The bisection method starts by picking (through inspection) two values of x , say, x_0 and x_1 such that $f(x_0)$ and $f(x_1)$ have opposite signs. Next, f is evaluated at the midpoint i.e., $x_2 = (x_0 + x_1)/2$ and depending on the sign of the evaluation, the search for the solution is narrowed to either between x_0 and x_2 or between x_2 and x_1 . The procedure continues recursively until an x is found such that $|f(x)| \leq 2^{-17}$. We will consider this x to be a solution. Print the solution and the number of calls (including the root call) to the recursive function needed to converge to the solution when $f(x) = x^3 + x^2 + x - 1.5$. You can assume that x is a single-precision floating-point number. SPIM implements the IEEE 754 standard for representing floating-point numbers.

Note: Feel free to define your own argument passing convention for floating-point arguments to functions. The expected root is 0.69141388 (the other two roots are complex). Please refer to Appendix A.10 of the text book for the details of floating-point instructions. The output should be in the following format.

```
Solution: 0.69141388
Number of calls: 17
```

3. Write a MIPS program to evaluate the trigonometric function $\tan(x)$ where x is input from keyboard. Assume that x is input in degrees and is a single-precision floating-point number. You can evaluate $\tan(x)$ by using its Maclaurin series or by evaluating $\sin(x)$ and $\cos(x)$ using their Maclaurin series. Remember that all these series are defined for x in radians. Print the value of $\tan(x)$.

Note: 32-bit integers can accommodate $n!$ for $n \leq 12$. On the other hand, if $n!$ is computed as a product of n single-precision floating-point numbers (i.e., $1.0, 2.0, \dots, n$) and stored as a single-precision floating-point number, one can go up to a much larger n offering much better accuracy for the calculated value of $\tan(x)$. The IEEE 754 single-precision representation of π is 0x40490fdb.